24 Hour C# .NET API Challenge / Project

GOAL

The goal of this test is to create a consumable RESTful API for importing, storing and retrieving sales data.

OVERVIEW

This test is built to help us assess three aspects:

1. How you manage to import a large amount of data from a CSV file
2. How you design / build a database schema to store the imported CSV data
3. How you approach designing a RESTful API. We are interested in how you organize code. While this application is trivial, we'd like to see it structured as if it would become an enterprise application.
4. Your familiarity with the current C# / .Net ecosystem.
5. Your use of C# .Net in in modern codebases.

PREREQUISITES

Use this dataset for the CSV import in the database that you will build along with the API for it.
A year's worth of sales from a fictitious pizza place, including the date and time of each order and the pizzas served, with additional details on the type, size, quantity, price, and ingredients.
LINK:

REQUIREMENTS

You are allowed to utilize the following programming language, frameworks and libraries:

1. C# .Net
2. Entity Framework

You may use other supporting libraries to implement the features required in this challenge.

Use Case

1. Create a public github repository to contain your work.
    a. Follow what you understand to be best git development practices
    b. Share this repository with us upon completion of the exercise
2. Build a simple backend application that supports the following oversimplified use-cases
    a. You can import the data from CSV file into your database schema
    b. Data is stored in a structured way in a new database schema which is suitable for the CSV file data
        i. Please also explain why you have chosen this schema and how it is suitable for the imported data
    c. Create a simple REST API to consume the data. You are free to design your own schema for the REST API.
        i. Have the API return JSON data
    d. Bonus: This is optional, but we do value your ideas for additional insights into the sales data. What statistics would you like to produce which might be important for the

business. More extra points for implementing your ideas and making these insights available via API endpoints.

24 Hour C# .NET API Challenge Scoring Rubric (Total: 100 Points)

Architecture & Code Organization – 25 pts

Criteria	Description	Points
Project Structure	Clean, modular folder and namespace organization (e.g., separation of concerns for Controllers, Services, Repositories, Models).	5
OOP Principles	Proper use of abstraction, encapsulation, inheritance, and polymorphism where applicable.	5
SOLID Principles	Code demonstrates Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.	10
Dependency Injection	Uses DI properly (via .NET built-in DI or other frameworks).	5

Data Management & Schema Design – 20 pts

Criteria	Description	Points
Schema Design	Logical and normalized structure reflecting the CSV data relationships (e.g., pizzas, ingredients, orders, customers).	8
Entity Framework Usage	Proper use of EF Core entities, configurations, and migrations. Avoids anti-patterns like heavy logic in DbContext.	6
Data Import Implementation	Efficient and fault-tolerant CSV import process (handles large files, validation, duplicates, and errors gracefully).	6

API Design & Implementation – 25 pts

Criteria	Description	Points
RESTful Principles	Follows correct use of HTTP verbs, routes, and response codes.	7
Consistency & Naming	Clear, consistent, and meaningful endpoint names and data models.	5
Data Retrieval	Correct implementation of pagination, filtering, and sorting where appropriate.	5
Error Handling & Validation	Properly handles invalid requests, missing data, and exceptions using middleware or filters.	5
JSON Responses	Returns well-structured JSON responses.	3

Code Quality & Best Practices – 15 pts

Criteria	Description	Points
Clean Code	Readable, maintainable code; meaningful variable/method names; comments only where needed.	5
Asynchronous Programming	Proper use of async/await for I/O operations (database and file handling).	5
Error & Exception Management	Centralized exception handling; avoids swallowing exceptions.	5

Performance, Testing & Extras – 15 pts

Criteria	Description	Points
Performance Considerations	Efficient queries, bulk imports, caching (if relevant).	5
Testing	Presence of unit/integration tests or testability by design.	5
Bonus Insights / Analytics	Additional endpoints or logic for business insights (e.g., top-selling pizzas, revenue trends).	5

Git & Delivery – 10 pts

Criteria	Description	Points
Commit Practices	Frequent, well-described commits reflecting logical work increments.	5
Repository Organization	Clear README with setup instructions, schema explanation, and API documentation.	5

Total: 100 Points
Passing Threshold: 70
Excellent Submission: 85+